

Backend do BdayList — documentação

Decisões e specs para pôr o BdayList no ar. Resumo executivo: **SPA static export + Supabase Cloud (Postgres + RLS + Edge Functions), região São Paulo (sa-east-1)**, MVP bootstrapped. O backend dedicado foi comparado e fica como rota futura sob gatilhos específicos.

ADRs (decisões)

#	Documento	O que decide
0001	Plataforma de backend	Nuvem, stack, BaaS vs backend dedicado, tradeoffs (com dados) + qual modelo de IA por agent
0002	Escolha de banco	Postgres vs MySQL vs MongoDB; reserva sem duplicidade; managed Postgres
0003	Segurança e LGPD	Threat model, RLS, anti-SSRF/XSS, LGPD, hardening priorizado

Specs (implementação)

#	Documento
01	Modelo de dados & contratos
02	Motor de reserva — o coração
03	Autorização & privacidade
04	Integrações — enrich de link, e-mail
05	Contratos de endpoints/RPCs

Skills (para os agents)

- `.claude/skills/bdaylist-backend/` — implementar/revisar features de backend
- `.claude/skills/bdaylist-rls-seguranca/` — RLS, segurança e LGPD

PDFs

Versões em PDF (simples de ler) em `pdf/`. Para regerar: `node docs/backend/pdf/gerar-pdfs.mjs`.

TL;DR da decisão

- Banco:** PostgreSQL. Reserva sem duplicidade por índice único parcial; privacidade por RLS fail-closed; metadados em JSONB.
- Nuvem:** Supabase Cloud, São Paulo. US\$0 em validação → US\$25/mês previsível.
- Backend dedicado:** só quando a regra não couber em RLS/RPC, entrar dinheiro, custo cruzar o tier Team, ou precisar de API pública versionada.
- Modelo de IA dos agents:** `opus` para arquitetura/decisão (`arq` , `dba` , `redteam`), `sonnet` para execução, `haiku` para tarefas baratas.